Exercises · Working With Claude

AI03 Exercises

Work each one before reading its solution. Exercises 1 and 4 are arithmetic. Practise Exercise 6 out loud. Prices and multipliers were checked in November 2026, so recheck them before relying on them.

Exercise 1: Fix a noisy reviewer

Your PR bot produces these results over a sample:

Category	Reported	Useful
Security	20	16
Logic bugs	30	21
Stale comments	50	5
Naming	100	8

(a) Compute precision by category and overall. **(b)** If you disable the two noisy categories, what's the new precision? How many useful findings do you lose? **(c)** Write one categorical criterion for stale comments. **(d)** Why not add "only report when confident"?

Solution

(a) Security 80%, logic 70%, stale comments 10%, naming 8%. Overall 50 useful of 200 = **25%**.

(b) 37 useful of 50 = **74%**. You lose 13 useful findings out of 50.

(c) "Flag a comment only when the behavior it claims contradicts what the code actually does. Don't flag comments that are merely incomplete or could be reworded."

(d) Self-reported confidence isn't calibrated, and the phrase doesn't define what counts. TS 4.1 names it as ineffective.

Exercise 2: Design an extraction schema

You extract fields from supplier contracts. Fields: supplier name, contract value, renewal date, governing law, and termination type (`for_cause`, `for_convenience`, or something else). Many contracts have no renewal date, and contract values may be stated only in an annex.

(a) Which fields should be nullable? **(b)** Design the termination type. **(c)** Which checks belong in code? **(d)** The contract value is only in an annex you don't have. Retry or not?

Solution

(a) Renewal date, contract value and governing law could all be absent. Supplier name defines the document, so it can be required.

(b) An enum of `for_cause`, `for_convenience`, `other` and `unclear`, plus `termination_detail` for `other`.

(c) Value not negative, renewal date after the start date, currency code format, and `termination_detail` present when the type is `other`. Strict schemas don't support numeric limits anyway.

(d) Don't retry. The information is absent. Return null, and flag it for a person, or fetch the annex.

Exercise 3: Find the invalidators

```
String system = "Today is " + LocalDate.now() + ".\n"
        + "Customer: " + customer.name() + "\n"
        + toJson(new HashMap<>(settings)) + "\n"
        + policyManual;
List<Tool> tools = toolsFor(customer.tier());
```

(a) List four things preventing good cache hit rates. **(b)** Rewrite the structure. **(c)** How do you confirm it worked?

Solution

(a) (1) The date changes daily, and it's at the start. (2) The customer name makes each prefix unique to one customer. (3) `HashMap` JSON may have varying key order. (4) Tools vary by tier, and tools render first, invalidating everything after them.

(b) Use a fixed tool list, sorted by name. Put the manual and stable settings, serialized with sorted keys, in the system prompt with a breakpoint. Put the date, the customer and the tier in a later message. Enforce tier rules in code rather than by hiding tools, or use a small number of fixed tool sets.

(c) Check `cache_read_input_tokens` on repeated requests, and alert when the hit rate falls. Add a unit test asserting that the prefix bytes are the same across requests.

Exercise 4: Batch planning

You analyze 300,000 documents nightly. Results are due within 28 hours of arrival. Each request has 4,000 input and 600 output tokens on Opus 5.5 ($4 and $20 per million).

(a) How many batches do you need at minimum, given the limit per batch? **(b)** What's the longest safe submission gap? **(c)** Compute synchronous and batch cost. **(d)** 2% fail, half with invalid requests, and half expired. What do you do with each?

Solution

(a) By request count, 300,000 / 100,000 = 3 batches. But check the other limit: 256 MB across 100,000 requests allows only about 2.6 KB each. At a rough 4 characters per token, a 4,000-token document is about 16 KB, so one batch holds about 16,000 of them. **Size decides, so you'd need about 19 batches.**

(b) gap + 24 ≤ 28, so at most **4 hours**. Use less, for a margin.

(c) Per request: (4,000 × $4 + 600 × $20) / 1M = $0.028. × 300,000 = **$8,400** synchronous, and **$4,200** by batch.

(d) 6,000 fail. The 3,000 invalid requests need fixing before they're resent, because they'd fail again. The 3,000 expired ones can be resubmitted, and you should investigate capacity and timing. Neither group is billed. Use `custom_id` for both.

Exercise 5: Assemble the context

A support session has reached turn 60. It has: a 2,000-token system prompt, 45 lookups of 800 tokens each, and a transcript of 30,000 tokens. The customer has stated a $340 refund expectation and a date of 15 November.

(a) Calculate the tool result tokens now, and after trimming to 100 tokens each. **(b)** Lay out the layers of your next request, in order. **(c)** What happens to the $340 under summarization, and how do you protect it? **(d)** Name one risk of trimming.

Solution

(a) 45 × 800 = **36,000**, which falls to 45 × 100 = **4,500**. That saves 31,500 tokens on every later turn.

(b) System prompt and tools first (stable, and cached), then a summary of old turns, then case facts, then recent turns, with the current question last. Case facts sit after the breakpoint if they change often.

(c) It may become "a refund". Put the amount and date in case facts, which stay outside the summary.

(d) Removing a field the agent needs later means refetching or a wrong answer. Also, removing source metadata destroys attribution. Review by role, and offer a `fields` parameter.

Exercise 6: Defend a decision

Your manager says: "Structured outputs guarantee correct JSON, so we can remove the validation layer and the human review queue. It'll cut costs by 40%." Write the answer you'd give out loud.

Model answer

"The guarantee is real, and we should use it. We can delete our JSON repair code and most format retries, and that saving is available right away.

But the guarantee is about shape, not truth. A schema can't tell that line items don't add up, and if a field is required, the model is under pressure to invent a value. So validation is what catches wrong meaning, and it's cheap, since it's only code. I'd keep it.

The review queue is the real cost, so let's reduce it using evidence. I'd sample accepted documents, stratified by document type, and measure field-level accuracy. Where a segment is reliably accurate, we can reduce review. Where it isn't, the savings would turn into payment errors.

I'd change my view if a segment holds its accuracy over a sustained, stratified sample. Then we can remove review for that segment, and keep sampling. That gives us much of the saving, without betting the ledger on an average."

Why this works: it accepts the valid part, separates syntax from meaning, proposes a measured path to the same goal, and says what would change your mind.

Your notes